C:\Windows\system32\cmd.exe

```
Microsoft Windows [Version 10.0.19045.6332]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Personal Computer>cd my-first-api

C:\Users\Personal Computer\my-first-api>npm init -y
Wrote to C:\Users\Personal Computer\my-first-api\package.json:

{
  "name": "my-first-api",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "express": "^5.2.1"
  },
  "devDependencies": {}
}


C:\Users\Personal Computer\my-first-api>
```

Lab 3.pdf

Lab 3. Setup REST API Server

Objective: Shift from being the "Client" to being the "Server." Set up a basic Node.js API that handles HTTP verbs (GET and POST).

Instructions

1. Setup Your Environment
   - Create a new folder named my-first-api (or rename something else) and navigate into it using your terminal.
   - Initialize a new Node.js project by running npm init -y.
2. Install Express
   - Express is a web framework that makes building APIs in Node.js incredibly simple. Install it by running
     - npm install express
3. Build the Server
   - Create a file named server.js (again, you can rename this all you want). Use the following hints to write the server logic yourself
     - Initialization: Require the express module, create your app instance, and define a constant for your PORT (e.g., 3000).
     - Middleware: Before you write your endpoints, you need to tell Express how to read JSON payloads. Use app.use() to implement the built-in express.json() middleware.
     - The GET Endpoint (Read Data): Create a route that listens for GET requests at the endpoint /api/greeting. Inside the callback function (req, res), use the response object to send a 200 status code and a JSON object with a welcome message (e.g., "Hello! Your API is working.").
     - The POST Endpoint (Receive Data): Create a route that listens for POST requests at the endpoint /api/echo.
       - Hint: You can access the data the client sent you via req.body. Store this in a variable.
       - Use the response object to send back a 201 status code (which means "Created") and a JSON object that includes both a success message and the data the user just sent you.
     - Start Listening: Finally, tell your app to listen on your defined PORT. Include a

C:\Windows\system32\cmd.exe - node server.js

```
Microsoft Windows [Version 10.0.19045.6332]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Personal Computer>cd my-first-api

C:\Users\Personal Computer\my-first-api>npm init -y
Wrote to C:\Users\Personal Computer\my-first-api\package.json:

{
  "name": "my-first-api",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "express": "^5.2.1"
  },
  "devDependencies": {}
}


C:\Users\Personal Computer\my-first-api>npm install express

up to date, audited 66 packages in 1m

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\Personal Computer\my-first-api>node server.js
Server is running on http://localhost:3000
```

Lab 3.pdf

Lab 3. Setup REST API Server

Objective: Shift from being the "Client" to being the "Server." Set up a basic Node.js API that handles HTTP verbs (GET and POST).

Instructions

1. Setup Your Environment
   - Create a new folder named my-first-api (or rename something else) and navigate into it using your terminal.
   - Initialize a new Node.js project by running npm init -y.
2. Install Express
   - Express is a web framework that makes building APIs in Node.js incredibly simple. Install it by running
     - npm install express
3. Build the Server
   - Create a file named server.js (again, you can rename this all you want). Use the following hints to write the server logic yourself
     - Initialization: Require the express module, create your app instance, and define a constant for your PORT (e.g., 3000).
     - Middleware: Before you write your endpoints, you need to tell Express how to read JSON payloads. Use app.use() to implement the built-in express.json() middleware.
     - The GET Endpoint (Read Data): Create a route that listens for GET requests at the endpoint /api/greeting. Inside the callback function (req, res), use the response object to send a 200 status code and a JSON object with a welcome message (e.g., "Hello! Your API is working.").
     - The POST Endpoint (Receive Data): Create a route that listens for POST requests at the endpoint /api/echo.
       - Hint: You can access the data the client sent you via req.body. Store this in a variable.
       - Use the response object to send back a 201 status code (which means "Created") and a JSON object that includes both a success message and the data the user just sent you.
     - Start Listening: Finally, tell your app to listen on your defined PORT. Include a

C:\Windows\system32\cmd.exe - node server.js

```
Microsoft Windows [Version 10.0.19045.6332]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Personal Computer>cd my-first-api

C:\Users\Personal Computer\my-first-api>npm init -y
Wrote to C:\Users\Personal Computer\my-first-api\package.json:

{
  "name": "my-first-api",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "express": "^5.2.1"
  },
  "devDependencies": {}
}



C:\Users\Personal Computer\my-first-api>npm install express

up to date, audited 66 packages in 1m

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\Personal Computer\my-first-api>node server.js
Server is running on http://localhost:3000
```

localhost:3000/api/greeting

Pretty-print

```
{"message":"Hello! Your API is working."}
```

C:\Windows\system32\cmd.exe - node server.js

```
Microsoft Windows [Version 10.0.19045.6332]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Personal Computer>cd my-first-api

C:\Users\Personal Computer\my-first-api>npm init -y
Wrote to C:\Users\Personal Computer\my-first-api\package.json:

{
  "name": "my-first-api",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
```

C:\Windows\system32\cmd.exe

```
Microsoft Windows [Version 10.0.19045.6332]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Personal Computer>cd my-first-api

C:\Users\Personal Computer\my-first-api>curl -X POST http://localhost:3000/api/echo -H "Content-Type: application/jso
n" -d "{\"name\":\"Blue\",\"role\":\"color\"}"
{"message":"Data received successfully.","data":{"name":"Blue","role":"color"}}
C:\Users\Personal Computer\my-first-api>
```

localhost:3000/api/greeting

Pretty-print

```
{"message":"Hello! Your API is working."}
```

server - Notepad

File Edit Format View Help

```
const express = require('express');

const app = express();
const PORT = 3000;

app.use(express.json());

app.get('/api/greeting', (req, res) => {
	res.status(200).json({
		message: 'Hello! Your API is working.'
	});
});

app.post('/api/echo', (req, res) => {
	const receivedData = req.body;

	res.status(201).json({
		message: 'Data received successfully.',
		data: receivedData
	});
});

app.listen(PORT, () => {
	console.log(`Server is running on http://localhost:${PORT}`);
});
```

MEMBERS:

MARY TRISHIA TAMAÑO

DARRYL ALTERADO

JAIZA SEGUEINTE

THERESE ANNE BALADIANG

MARY CHRIS NAVAROZA